



LIONSDEV

Curso de Programação

MÓDULO 2

Fundamentos de Programação



Professor Nicolas Cardoso Motta

Revisando o Módulo 1



Node.js

Roda JavaScript fora do navegador: **node** **arquivo.js** executa um arquivo.



Terminal

pwd, **ls**, **cd** e **mkdir**: saiba onde você está antes de rodar qualquer coisa.



npm

npm init -y cria o **package.json**; **npm install** baixa as dependências.

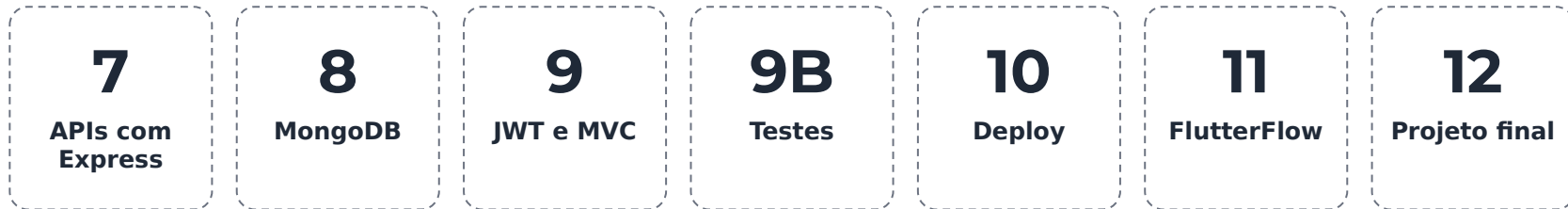
Pergunta para abrir: o terminal respondeu Cannot find module. O que você confere primeiro?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND



FRONTEND E PROJETO

Você está aqui: **Módulo 2**, Fundamentos de Programação e Debugging com Node.js

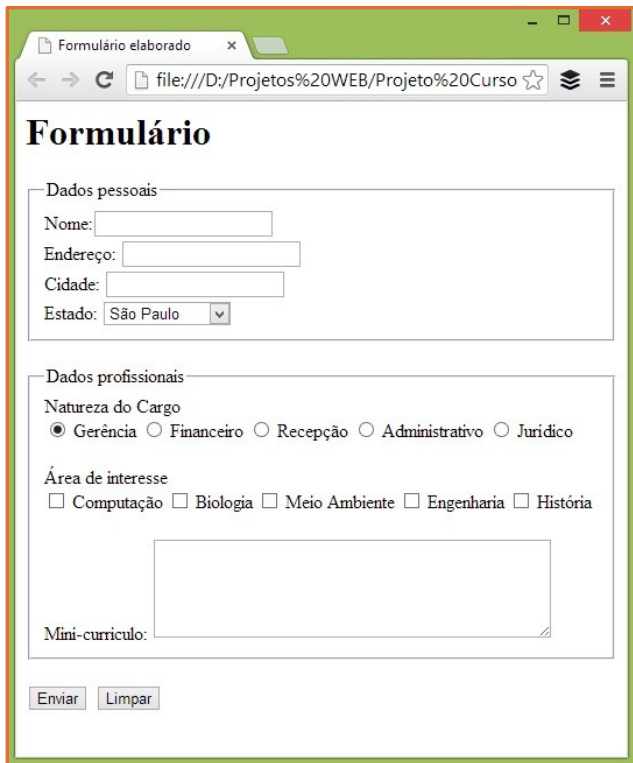


Objetivo

- Entender o que são JavaScript e Node.js e onde cada um roda.
- Criar variáveis e trabalhar com tipos de dados, operadores e entrada e saída no terminal.
- Controlar o fluxo do programa com condicionais, laços, switch e funções.
- Organizar informações com arrays e objetos.
- Encontrar e corrigir erros usando o debugger do VS Code.

No módulo anterior preparamos o ambiente de desenvolvimento. Agora vamos voltar ao JavaScript e ao Node.js para escrever nossos primeiros programas de verdade.

Um pouco de história



The screenshot shows a Netscape browser window with a single tab titled 'Formulário elaborado'. The address bar shows a local file path: 'file:///D:/Projetos%20WEB/Projeto%20Curso'. The form itself is titled 'Formulário' and is divided into two main sections: 'Dados pessoais' and 'Dados profissionais'. The 'Dados pessoais' section contains input fields for 'Nome:', 'Endereço:', 'Cidade:', and a dropdown menu for 'Estado:' (currently set to 'São Paulo'). The 'Dados profissionais' section includes a 'Natureza do Cargo' section with radio buttons for 'Gerência' (selected), 'Financeiro', 'Recepção', 'Administrativo', and 'Jurídico'. Below this is an 'Área de interesse' section with checkboxes for 'Computação', 'Biologia', 'Meio Ambiente', 'Engenharia', and 'História'. At the bottom of the form is a large text area labeled 'Mini-curriculo:' and two buttons: 'Enviar' and 'Limpar'.



Surgiu em 1995

Criada por Brendan Eich, na Netscape, em cerca de 10 dias.



Validar formulários

O objetivo era ser uma linguagem fácil para validar formulários HTML direto no navegador.



Revolucionária

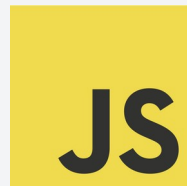
Pela primeira vez, as páginas podiam reagir ao usuário sem recarregar.

JavaScript não é Java



Java

Linguagem muito popular em 1995. O nome "JavaScript" foi uma jogada de marketing para pegar carona nessa fama.



JavaScript

Criada para o navegador. Apesar do nome parecido, funciona de um jeito completamente diferente do Java.

Java está para JavaScript assim como carro está para carpete.

O que é JavaScript?



A linguagem da web

Originalmente usada para dar interatividade aos sites: botões, formulários, animações.



Roda no navegador

As únicas linguagens que o navegador executa diretamente são JavaScript e WebAssembly.



Interpretada e de alto nível

Você escreve algo próximo da linguagem humana e o interpretador traduz para a máquina.

Curiosidade: isso é Assembly

```
000000000001169 <main>:
1169: f3 0f 1e fa      endbr64
116d: 55              push  %rbp
116e: 48 89 e5        mov   %rsp,%rbp
1171: ba 0c 00 00 00  mov   $0xc,%edx
1176: 48 8d 35 87 0e 00 00  lea   0xe87(%rip),%rsi      # 2004 <_IO_stdin_used+0x4>
117d: bf 01 00 00 00  mov   $0x1,%edi
1182: e8 d9 fe ff ff  callq 1060 <write@plt>
1187: bf 00 00 00 00  mov   $0x0,%edi
118c: e8 df fe ff ff  callq 1070 <exit@plt>
1191: 66 2e 0f 1f 84 00 00  nopw  %cs:0x0(%rax,%rax,1)
1198: 00 00 00
119b: 0f 1f 44 00 00  nopl  0x0(%rax,%rax,1)

0000000000011a0 <_libc_csu_init>:
11a0: f3 0f 1e fa      endbr64
11a4: 41 57          push  %r15
11a6: 4c 8d 3d 03 2c 00 00  lea   0x2c03(%rip),%r15      # 3db0 <__frame_dummy_init_array_entry>
11ad: 41 56          push  %r14
11af: 49 89 d6        mov   %rdx,%r14
11b2: 41 55          push  %r13
11b4: 49 89 f5        mov   %rsi,%r13
11b7: 41 54          push  %r12
11b9: 41 89 fc        mov   %edi,%r12d
11bc: 55            push  %rbp
11bd: 48 8d 2d f4 2b 00 00  lea   0x2bf4(%rip),%rbp      # 3db8 <__do_global_ctors_aux_fini_array_entry>
11c4: 53            push  %rbx
11c5: 4c 29 fd        sub   %r15,%rbp
11c8: 48 83 ec 08     sub   $0x8,%rsp
11cc: e8 2f fe ff ff  callq 1000 <_init>
11d1: 48 c1 fd 03     sar   $0x3,%rbp
11d5: 74 1f          je    11f6 <_libc_csu_init+0x56>
11d7: 31 db          xor   %ebx,%ebx
11d9: 0f 1f 80 00 00 00 00  nopl  0x0(%rax)
11e0: 4c 89 f2        mov   %r14,%rdx
11e3: 4c 89 ee        mov   %r13,%rsi
11e6: 44 89 e7        mov   %r12d,%edi
11e9: 41 ff 14 df     callq *(%r15,%rbx,8)
11ed: 48 83 c3 01     add   $0x1,%rbx
11f1: 48 39 dd        cmp   %rbx,%rbp
11f4: 75 ea          jne   11e0 <_libc_csu_init+0x40>
11f6: 48 83 c4 08     add   $0x8,%rsp
11fa: 5b            pop   %rbx
11fb: 5d            pop   %rbp
11fc: 41 5c          pop   %r12
11fe: 41 5d          pop   %r13
1200: 41 5e          pop   %r14
1202: 41 5f          pop   %r15
1204: c3            retq
1205: 66 66 2e 0f 1f 84 00  data16 nopw %cs:0x0(%rax,%rax,1)
```

Baixo nível

Cada linha é uma instrução quase direta para o processador.

Alto nível

No JavaScript, um simples `console.log("Oi")` faz todo esse trabalho por você.



Como um site é desenvolvido?

Um site é como uma casa



HTML

**A estrutura:
paredes, portas e
janelas. Define o que
existe na página.**



CSS

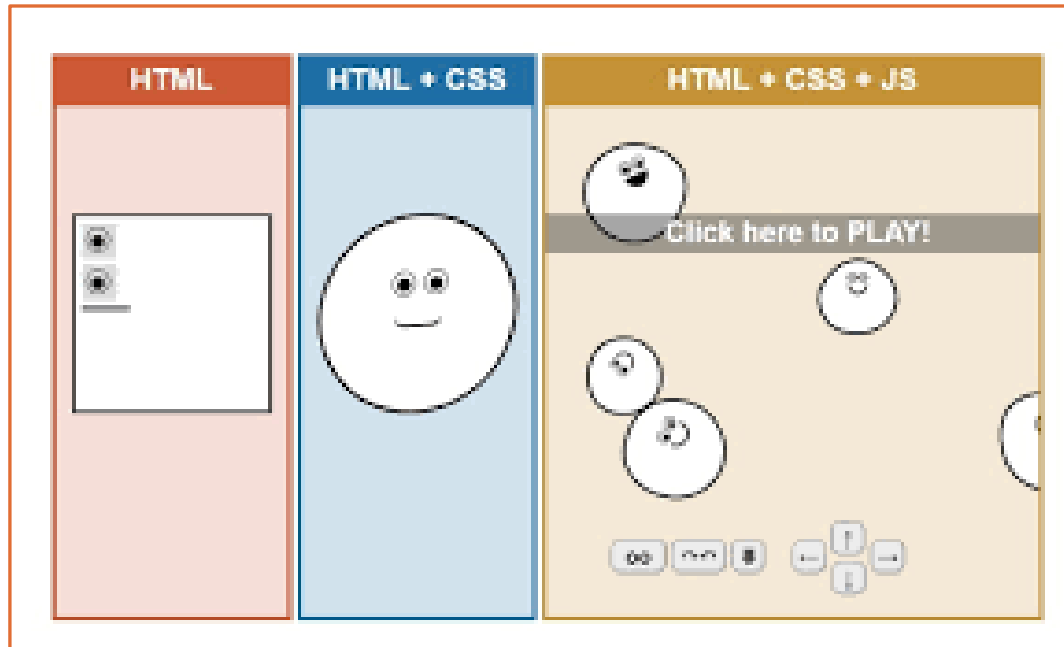
**A pintura e a
decoração: cores,
fontes e layout.
Deixa tudo bonito.**



JavaScript

**A eletricidade e o
encanamento: dá
vida à casa com
interatividade.**

Na prática



Só HTML: funciona, mas é cru. Com CSS: fica bonito. Com JavaScript: ganha vida.

JavaScript está em todo lugar



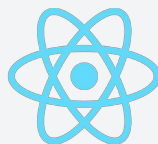
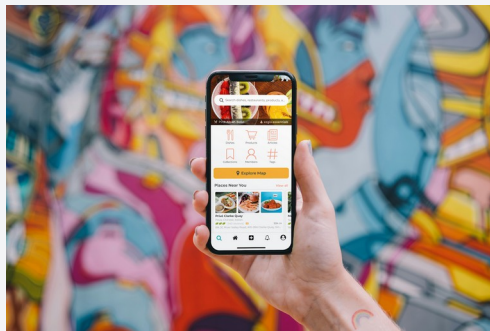
"Tudo que puder ser escrito com JavaScript, eventualmente será escrito com JavaScript."

— Lei de Atwood

Onde o JavaScript roda hoje



Servidores
Node.js



Celulares
React Native



Desktop
Electron (ex.: VS Code)

Node.js



JavaScript fora do navegador

Permite executar JavaScript no servidor, indo além do contexto tradicional do navegador.



Ambiente de execução

É ele que executa (interpreta) o código que vamos escrever, usando o motor V8 do Google Chrome.



npm

Milhares de pacotes prontos, instalados com um comando: `npm install nome-do-pacote`.

JavaScript x Node.js



Linguagem de programação

Criada para ser usada no navegador.

Muito usada no front-end.



Ambiente de execução

Permite rodar JavaScript fora do navegador.

Muito usado no back-end.

Comparar os dois é como comparar maçãs e laranjas: um é a língua, o outro é onde ela é falada.



Como um programa funciona?

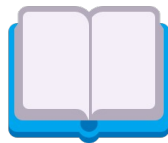
Entrada, processamento e saída

Voltando ao exemplo da cozinha...



Entrada

Os ingredientes: leite, ovos, farinha...



Programa

A receita: a sequência de passos que transforma a entrada.



Saída

O bolo pronto: o resultado que queríamos.

Todo programa recebe dados, processa esses dados e devolve um resultado.

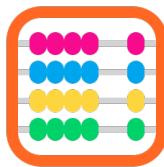
Exemplo: média de três notas

Um programa que calcula a média dos alunos:



Entrada

As três notas: **nota1**, **nota2** e **nota3**.



Programa

Soma as três notas e divide o total por 3.



Saída

O **resultado**: a média do aluno.

soma, nota1, nota2, nota3 e resultado
guardam valores enquanto o programa
roda. Todos eles são **variáveis**.



Variáveis

O que são e como criar variáveis em JavaScript

O que são variáveis?



Uma caixa com etiqueta

Um espaço na memória do computador onde guardamos um valor para usar depois.



Têm um nome

O nome é o identificador que usamos para acessar o valor de forma fácil.



O valor pode mudar

Podemos ler, alterar e reaproveitar o valor durante a execução do programa.



Têm um tipo

Cada valor tem um tipo de dado: texto, número, verdadeiro/falso...



```
let idade = 20
```

Três formas de criar uma variável

var

Forma antiga.
Escopo de função;
pode ser
redeclarada.
Evite em código
novo.

```
var nome = 'João'
```

let

Escopo de bloco { }.
O valor pode ser
alterado.

```
let nome = 'João'
```

const

Escopo de bloco { }.
O valor não pode ser
reatribuído.

```
const nome = 'João'
```

Regra prática: use const por padrão e let quando o valor precisar mudar.

let muda, const não

```
let idade = 20
idade = 21           // ok: let pode mudar

const pais = 'Brasil'
pais = 'Argentina'  // erro!
```

let → aceita um novo valor a qualquer momento.

const → tentar trocar o valor gera **TypeError: Assignment to constant variable**.

dica → se o valor não vai mudar (um nome, o ano atual, uma taxa), use **const**.



Cuidado: let não se declara duas vezes

Criar de novo uma variável que já existe dá erro, e o programa **nem roda.**

- Acontece muito quando resolvemos várias atividades no mesmo arquivo: cada uma cria **let nome** de novo.
- Para mudar o valor, escreva só **nome = ...**, sem o **let**. Ou crie um arquivo por atividade.

```
let nome = 'Ana'
let nome = 'Bia'    // SyntaxError: Identifier 'nome' has already been declared
nome = 'Bia'        // correto: só troca o valor
```

Muito cuidado com os nomes!

Evite

Nomes genéricos ou abreviações que ninguém entende.

```
let x = 18  
let n1 = 'Ana'  
let dt = 2024
```

Prefira

Nomes descritivos, que dizem o que a variável guarda.

```
let idadeMinima = 18  
let nomeAluno = 'Ana'  
let anoAtual = 2024
```

Bons nomes facilitam a leitura para você e para quem vai revisar ou dar manutenção no seu código.

Convenções de nomenclatura



camelCase

Primeira letra minúscula e a inicial das outras palavras maiúscula: **nomeCompleto**. É o padrão para variáveis e funções em JavaScript.



PascalCase (UpperCamelCase)

Todas as iniciais maiúsculas: **NomeCompleto**. Usado em classes.



snake_case

Underscore no lugar dos espaços: **nome_completo**. Comum em Python e bancos de dados.

Nomes de arquivos: terminam em .js e não usam acentos nem espaços (ex.: saudacao.js).

Agora vamos executar nosso primeiro código!

A tradição: Hello World!



O primeiro programa

Em quase todas as linguagens, o primeiro programa exibe "Hello World" (Olá, Mundo).

```
console.log("Olá, mundo!")
```



Curiosidade

Diz a lenda que quem não começa por ele terá má sorte aprendendo a linguagem!

Saída no terminal

```
$ node teste.js  
Olá, mundo!
```



Objetivo

Fazer o computador exibir uma mensagem com `console.log()`.



Hora da Prática

Atividade Prática 1

Primeiro código

1. No VS Code, abra o arquivo que você criou (ou crie um chamado `teste.js`).
2. Escreva o comando para exibir a frase: "Estou programando em JavaScript!".
3. Salve o arquivo.
4. No terminal, execute usando o Node: `node teste.js`.



Desafio: tente imprimir o resultado de uma conta, como `console.log(10 + 5)`. O que acontece?

Atividade Prática 2

Variáveis

- **Crie três variáveis: nomeCurso (texto), anoAtual (número) e estaAtivo (verdadeiro ou falso).**
- **Imprima cada uma no console com uma mensagem explicando o que ela guarda.**



Dica: use `let` para as que podem mudar e `const` para as que não mudam.



Desafio: tente alterar o valor de uma variável criada com `const` e veja o erro que aparece.

Atividade Prática 3

Variáveis

- Crie duas variáveis, a e b, com os valores 10 e 3.
- Imprima no console: a soma, a diferença, o produto, a divisão e o resto da divisão entre elas.



Dica: cada resultado pode ficar em uma variável própria antes do `console.log()`.



Desafio: troque os valores de a e b entre si usando uma terceira variável auxiliar.



Strings

Nosso primeiro tipo de dado. Alguém sabe o que é uma string?

O que é uma String?



Uma cadeia de caracteres

Em inglês, "string" é um cordão: uma sequência de letras, números e símbolos.

```
let nome = 'João'  
let mensagem = "Conte comigo!"  
let idade = '20' // também é texto!
```



Serve para guardar texto

Nomes, mensagens, endereços... tudo que é texto é string.



Sempre entre aspas

Aspas simples ' ' ou duplas " ". (A crase ` ` veremos logo adiante.)

Aspas dentro de strings

Se o texto precisa ter aspas, use o outro tipo de aspas por fora:

```
let exemplo1 = 'Isso é uma "string" com aspas duplas dentro.'  
let exemplo2 = "Isso é uma 'string' com aspas simples dentro."  
let exemplo3 = 'Use \' para escapar a mesma aspa.'
```

A barra invertida \ "escapa" o caractere: ele passa a fazer parte do texto.

Concatenação de Strings

Concatenar é juntar duas ou mais strings em uma nova, usando o operador **+**.

```
let nome = "João"  
let saudacao = "Olá, " + nome + "  
let fraseCompleta = saudacao + " Hoje é um belo dia."  
  
console.log(fraseCompleta)
```

Saída no terminal

```
$ node strings.js  
Olá, João! Hoje é um belo dia.
```

Interpolação com crase

Com a crase `` ``, colocamos variáveis direto dentro do texto usando `${ }`:

```
let nome = "João"  
let idade = 20  
  
let mensagem = `Olá, ${nome}! Você tem ${idade} anos.`  
console.log(mensagem)
```

Saída no terminal

```
$ node strings.js  
Olá, João! Você tem 20 anos.
```



Cuidado: interpolação só funciona com crase

Com aspas simples ou duplas, o **`${nome}`** é impresso exatamente do jeito que foi escrito.

- O **`${ }`** só é interpretado dentro da crase `` ``.
- Com aspas normais, o JavaScript entende tudo como texto comum.

```
let nome = 'Ana'  
console.log('Olá, ${nome}!') // Olá, ${nome}!  
console.log(`Olá, ${nome}!`) // Olá, Ana!
```

Métodos básicos de strings

```
let nome = ' Ana Souza '
```



```
nome.length           // 13 (os espaços contam!)  
nome.trim()           // 'Ana Souza'  
nome.toUpperCase()    // ' ANA SOUZA '  
nome.toLowerCase()    // ' ana souza '
```



```
let resposta = ' SIM '  
resposta.trim().toLowerCase() // 'sim'
```

length → quantos caracteres o texto tem. Espaços também contam.

trim() → tira os espaços do começo e do fim.

toUpperCase() → deixa tudo maiúsculo; **toLowerCase()**, tudo minúsculo.

encadear → dá para usar um depois do outro. Ótimo para comparar respostas do usuário.



Hora da Prática

Atividade Prática 1

Strings

1. Abra o terminal e navegue até a pasta do módulo dentro de Documentos.
2. Crie uma pasta chamada `variaveis` e abra o VS Code dentro dela.
3. Crie o arquivo `saudacao.js` e, nele, uma variável `nome` com o seu nome.
4. Crie uma variável `mensagem` com uma mensagem de `olá`.
5. Use concatenação para unir a mensagem e o nome no formato: `Olá, nome`.
6. Imprima com `console.log()` e execute o programa com o Node.js.

Atividade Prática 2

Strings

- **Crie as variáveis nome e sobrenome com seus próprios dados.**
- **Monte uma variável nomeCompleto juntando as duas com um espaço no meio, usando concatenação (+).**
- **Depois monte a mesma frase usando interpolação (crase e \${ }) e imprima as duas no console.**



Desafio: imprima quantas letras tem o seu nome completo usando `.length`.

Atividade Prática 3

Strings

- Crie uma variável `cidade` com o nome da sua cidade e outra `estado` com a sigla do estado.
- Imprima uma frase no formato: "Eu moro em `[cidade]`, `[estado]`." usando interpolação.



Dica: a crase (`) fica na tecla ao lado do 1 no teclado brasileiro.



Desafio: use `.toUpperCase()` para imprimir a sigla do estado em letras maiúsculas.



Interação com o Usuário

Até agora, os valores já estavam escritos nas variáveis. Como fazer o **usuário digitar algo (entrada) e o programa responder (saída)?**

Entrada de dados com prompt-sync



O programa passa a "ouvir"

Até agora o código era fixo. Agora o usuário digita e o programa usa a resposta.



A ferramenta

O pacote **prompt-sync**, instalado pelo npm.



Instalação (no terminal)

Dentro da pasta do projeto, rode os dois comandos ao lado.

```
$ npm init -y  
Wrote to package.json
```

```
$ npm install prompt-sync  
added 3 packages in 1s
```

Como trazer pacotes para o código?

CommonJS · require

O padrão tradicional do Node.js.
Funciona sem nenhuma configuração extra.

```
const prompt = require('prompt-sync')()
```

ES Modules · import

O padrão moderno. Precisa de
"type": "module" no package.json.

```
import promptSync from 'prompt-sync'  
const prompt = promptSync()
```

Neste módulo vamos usar require. As duas formas fazem a mesma coisa: importar uma ferramenta.

Usou import sem configurar? Olhe o terminal



Node.js recente: roda, mas avisa

Aparece um warning amarelo antes da saída do programa:

```
[MODULE_TYPELESS_PACKAGE_JSON] Warning:
Module type of ../index.js is not
specified and it doesn't parse as
CommonJS. Reparsing as ES module...
To eliminate this warning, add
"type": "module" to package.json.
```



Node.js antigo: nem roda

O programa para com erro:

```
Warning: To load an ES module,
set "type": "module" in the
package.json or use the .mjs
extension.
```

```
SyntaxError: Cannot use import
statement outside a module
```

Nos dois casos, a correção é a mesma: avisar ao Node, no package.json, que o projeto usa import.

Corrigindo no package.json

Antes

```
{
  "name": "variaveis",
  "version": "1.0.0",
  "main": "index.js",
  "dependencies": {
    "prompt-sync": "^4.2.0"
  }
}
```

Depois

```
{
  "name": "variaveis",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module",
  "dependencies": {
    "prompt-sync": "^4.2.0"
  }
}
```

Adicione a linha "type": "module", (com a vírgula no final), salve o arquivo e rode de novo: o aviso some.



Cuidado: com "type": "module", o require para

O projeto inteiro passa a usar **import**: qualquer **require** dá **ReferenceError**.

- Cada projeto usa um sistema só: ou **require** (CommonJS), ou **import** (ES Modules).
- Quer testar o import? Crie uma pasta nova, com o próprio package.json, sem mexer no projeto das atividades.

```
// package.json com "type": "module"  
const prompt = require('prompt-sync')()  
// ReferenceError: require is not defined in ES module scope
```

Exemplo de uso

```
// 1. Prepara a ferramenta
const prompt = require('prompt-sync')({ sigint: true })

// 2. Pergunta e guarda a resposta (sempre vira String!)
let nome = prompt('Qual seu nome? ')
let cidade = prompt('Onde você mora? ')

console.log('Olá, ' + nome + ', de ' + cidade + '!')
```

Saída no terminal

```
$ node prompt.js
Qual seu nome? Ana
Onde você mora? Ponta Grossa
Olá, Ana, de Ponta Grossa!
```

Com sigint: true, o Ctrl + C encerra o programa. Sem isso, o prompt devolve null e o programa continua rodando.



Hora da Prática

Atividade Prática 1

Entrada e saída

1. Inicialize o projeto com `npm init -y`.
2. Instale o `prompt-sync` na sua pasta: `npm install prompt-sync`.
3. Crie um arquivo `.js` e peça ao usuário o nome do pet e a idade do pet.
4. Exiba: "O [NOME] tem [IDADE] anos!".
5. Execute com o comando `node nomeDoArquivo.js`.

Atividade Prática 2

Entrada e saída

- Peça ao usuário o nome de um produto e o preço dele.
- Imprima no console: "O produto [nome] custa R\$ [preço]."



Cuidado: para fazer contas, converta o preço com `parseFloat()`. Se o usuário digitar vírgula (19,90), troque por ponto antes com `.replace(",", ".")`.



Desafio: pergunte também a quantidade e mostre o valor total da compra.

Atividade Prática 3

Entrada e saída

- Peça ao usuário o ano em que ele nasceu e calcule a idade dele.
- Imprima: "Você tem [idade] anos." — lembre-se de converter a entrada com `parseInt()`.



Dica: `idade = anoAtual - anoNascimento`



Desafio: informe também em quantos anos a pessoa completará 100 anos.



O usuário digita 10 e depois 5. O que aparece?

```
const prompt = require("prompt-sync")()  
  
let a = prompt("Primeiro número: ")  
let b = prompt("Segundo número: ")  
  
console.log(a + b)
```

Pense antes de rodar: qual é a saída?



Cuidado: tudo que entra pelo prompt é texto

Imprime **105**, e não 15.

- O prompt-sync sempre devolve String. Com texto no meio, o **+** deixa de somar e passa a juntar (concatenar).
- Converta antes da conta: **parseInt()** para inteiros, **parseFloat()** para decimais ou **Number()**.

```
let a = parseInt(prompt("Primeiro número: "))
let b = parseInt(prompt("Segundo número: "))
console.log(a + b)    // 15
```



Cuidado: aqui a vírgula não é decimal

Para o JavaScript, o separador decimal é o **ponto**. Quem digita **7,5** recebe 7 ou NaN.

- `parseFloat` lê até encontrar a vírgula e para ali: `parseFloat("7,5")` dá 7.
- `Number("7,5")` não entende o texto e devolve **NaN** (Not a Number).
- Solução: troque a vírgula por ponto antes de converter, com `.replace(", ", ".")`.

```
let entrada = prompt("Nota: ")           // "7,5"  
let nota = parseFloat(entrada.replace(", ", ".")) // 7.5
```

Além do prompt-sync, o Node.js tem uma forma **nativa de interagir com o usuário...**



Stdin e Stdout (extra)

Conteúdo extra: não será cobrado nas atividades nem na avaliação. É a forma nativa do Node.js de ler e escrever — vale conhecer, mas usaremos o prompt-sync.

Stdin e Stdout



stdin · entrada

Fluxo de entrada padrão, geralmente o teclado.

É o "ouvido" do computador: espera você apertar Enter e entrega o que foi digitado.



stdout · saída

Fluxo de saída padrão, geralmente o console.

É onde as mensagens do programa aparecem. O `console.log()` usa ele por baixo dos panos.

Exemplo com entrada e saída

```
console.log('Digite algo e pressione Enter:')

process.stdin.on('data', function (data) {
  let entrada = data.toString().trim()
  process.stdout.write('Olá, ' + entrada + '!\n')
  process.exit()
})
```

stdin.on('data') → escuta o que o usuário digita.

toString() → converte os dados recebidos (Buffer) em texto; **trim()** tira o Enter do final.

stdout.write → escreve sem pular linha; por isso o **\n**.

process.exit() → encerra o programa.

Atividade extra

Não será cobrada

1. Na pasta variaveis, crie um novo arquivo chamado boas-vindas.js.
2. Use o `console.log()` para perguntar o nome do usuário.
3. Usando o `stdin`, colete o nome digitado e guarde em uma variável `nome`.
4. Crie uma variável `saudacao` no formato "Olá, Nome".
5. Imprima a saudação no console e execute o programa com o `Node.js`.

Revisão



Variáveis

Caixas com nome que guardam valores.

```
const nome = 'Ana'  
let idade = 20
```



Strings

Texto entre aspas, que pode ser concatenado ou interpolado.

```
'Olá, ' + nome  
`Olá, ${nome}`
```



Entrada e Saída

O usuário digita, o programa responde.

```
prompt('Nome? ')  
console.log(nome)
```

Concatenação x Interpolação

Concatenação (+)

```
let nome = "João"  
let mensagem = "Olá, " + nome + "!"
```

Interpolação (crase e \${ })

```
let nome = "João"  
let mensagem = `Olá, ${nome}!`
```

As duas geram o mesmo texto: "Olá, João!". A interpolação fica mais legível quando há muitas variáveis.



Tipos de Dados

Tipos de dados em JavaScript



String

Texto: uma cadeia de caracteres.

```
let nome = 'João'
```



Number

Inteiros, decimais, NaN e Infinity.

```
let idade = 15  
let media = 7.5
```



Boolean

Verdadeiro (**true**) ou falso (**false**).

```
let portaAberta = false
```



Undefined

A variável existe, mas ainda não recebeu valor.

```
let aindaNaoSei
```



Null

Ausência de valor, colocada de propósito.

```
let vazio = null
```



Object

Coleção de propriedades: nome e valor.

```
let carro = { marca: 'Ford' }
```

Descobrimos o tipo com typeof

```
let aindaNaoSei

console.log(typeof 'João')    // string
console.log(typeof 15)        // number
console.log(typeof true)      // boolean
console.log(typeof aindaNaoSei) // undefined
console.log(typeof { a: 1 })  // object
console.log(typeof null)      // object (!)
```

typeof → devolve, em forma de texto, o tipo do valor.

typeof null → dá "object" por um erro histórico da linguagem, que nunca foi corrigido para não quebrar sites antigos.

'7' x 7 → com aspas é string; sem aspas é number. O **typeof** mostra a diferença.



Operadores

Operadores aritméticos

+ Adição

```
10 + 5 // 15
```

- Subtração

```
10 - 5 // 5
```

*** Multiplicação**

```
10 * 5 // 50
```

/ Divisão

```
10 / 5 // 2
```

% Módulo (resto)

```
10 % 3 // 1
```

Funcionam como na matemática. O % devolve o resto da divisão: ótimo para saber se um número é par.



Cuidado com a ordem das operações

Como na matemática, **multiplicação e divisão vêm antes de soma e subtração.**

- Sem parênteses, `nota1 + nota2 / 2` divide só a `nota2` e depois soma a `nota1`.
- Use parênteses para dizer o que deve ser calculado primeiro.

```
let nota1 = 7
let nota2 = 8
console.log(nota1 + nota2 / 2)    // 11    (errado: 7 + 4)
console.log((nota1 + nota2) / 2)  // 7.5    (correto)
```



Cuidado: linha que começa com (ou [

Sem ponto e vírgula, o Node **junta** a linha com a anterior quando ela começa com (ou [.

- O JavaScript coloca o ; sozinho no fim de cada linha, menos quando a próxima começa com (ou [: aí ele lê as duas como uma instrução só.
- No nosso estilo, sem ;, não comece uma linha com (ou [: guarde o valor numa variável ou coloque dentro do `console.log()`.

```
let soma = nota1 + nota2
(nota1 + nota2) / 2 // TypeError: nota2 is not a function

let media = (nota1 + nota2) / 2 // correto
```

Casas decimais: 0.1 + 0.2

```
console.log(0.1 + 0.2)           // 0.30000000000000004

let media = 20 / 3
console.log(media)                // 6.666666666666667
console.log(media.toFixed(2))    // 6.67 (é texto!)

let preco = 19.9
console.log(`R$ ${preco.toFixed(2)}`) // R$ 19.90
```

0.1 + 0.2 → 0
computador guarda decimais em binário, e alguns valores ficam com uma "sobra" minúscula. Não é bug do seu código.

toFixed(2) → arredonda
para 2 casas na hora de mostrar.

texto → o toFixed
devolve string: use só para exibir, não para continuar fazendo contas.



Cuidado: quando 10 + 5 vira 105

Se houver um **texto** no meio de uma soma, o JavaScript desiste da matemática e **gruda** os valores.

- É por isso que usamos `parseInt()`, `parseFloat()` ou `Number()` nas respostas do prompt-sync.
- Eles garantem que o texto vire número de verdade antes da conta acontecer.

```
"10" + 5           // "105"  texto + número = junta  
parseInt("10") + 5  // 15     agora sim, matemática
```



Hora da Prática

Atividade Prática 1

Operadores

- Usando entrada e saída, desenvolva um programa que peça ao usuário duas notas e calcule a média entre elas.



Cuidado: a nota pode ter casas decimais. Converta com `parseFloat()`, trocando a vírgula por ponto, e mostre a média com `.toFixed(1)`.

Atividade Prática 2

Tipos de dados

- Crie uma variável para cada tipo de dado que vimos: string, number, boolean e undefined.
- Use o operador `typeof` para imprimir o tipo de cada uma no console.



Dica: `console.log(typeof minhaVariavel)` mostra o tipo em forma de texto.



Desafio: crie uma variável com o valor "7" (com aspas) e outra com 7 (sem aspas) e compare os tipos.

Revisão



Strings

Concatenação com +
ou interpolação com
crase.

```
"Olá, " + nome  
`Olá, ${nome}`
```



Tipos

string, number,
boolean, undefined,
null e object.

```
typeof 15  
// 'number'
```



Operadores

Contas como na
matemática, e %
para o resto.

```
10 + 5 // 15  
10 % 3 // 1
```

Mais alguns operadores bem úteis...

Operadores de atribuição

Forma longa

```
let ano = 2023  
ano = ano + 1           // 2024
```

```
let alunos = 58  
alunos = alunos - 1    // 57  
alunos = alunos + 1    // 58  
alunos = alunos - 1    // 57
```

Forma curta

```
let ano = 2023  
ano += 1           // 2024
```

```
let alunos = 58  
alunos -= 1        // 57  
alunos++           // 58  
alunos--           // 57
```

Também existem *= e /=. O ++ e o -- somam ou subtraem exatamente 1.



Condicionais

Fazendo o programa tomar decisões

if, else if e else

if

Se

Se (alguma condição), então { uma ação }.

else

Senão

Se nenhuma condição for verdadeira, faça { uma ação geral }.

else if

Senão, se

Testa outra condição quando a anterior foi falsa. Pode haver quantos você precisar.

O JavaScript testa de cima para baixo e executa só o primeiro bloco cuja condição for verdadeira.

Condicionais na prática

```
let nota = 6

if (nota >= 7) {
  console.log('Aprovado')
} else if (nota >= 5) {
  console.log('Recuperação')
} else {
  console.log('Reprovado')
}
```

Saída no terminal

```
$ node condicionais.js
Recuperação
```

if (nota >= 7) → falso para 6: pula este bloco.

else if (nota >= 5) → verdadeiro: imprime "Recuperação" e para aqui.

else → só roda se todas as condições anteriores forem falsas.

Cuidado: variável criada dentro de um bloco

Assim dá erro

```
if (nota >= 7) {  
  let situacao = 'Aprovado'  
}  
  
console.log(situacao)  
// ReferenceError:  
// situacao is not defined
```

Assim funciona

```
let situacao  
if (nota >= 7) {  
  situacao = 'Aprovado'  
}  
  
console.log(situacao)  
// Aprovado
```

Com let ou const, a variável criada dentro de { } só existe dentro dessas chaves. Declare antes do bloco e atribua o valor dentro dele.



Operadores de Comparação

Eles comparam valores e sempre devolvem **true** ou **false**

Igualdade e diferença

== Igual (valor)

```
5 == 5    // true  
5 == '5'  // true
```

=== Igual (valor e tipo)

```
5 === 5    // true  
5 === '5'  // false
```

!= Diferente (valor)

```
5 != 6    // true  
5 != 5    // false
```

!== Diferente (valor e tipo)

```
5 !== '5' // true  
5 !== 5   // false
```

Não confunda com a atribuição (=). Prefira === e !==: eles não fazem conversões escondidas.

Maior e menor

> Maior

```
5 > 5    // false  
5 > 3    // true
```

< Menor

```
5 < 5    // false  
5 < 6    // true
```

>= Maior ou igual

```
5 >= 3    // true  
5 >= 5    // true
```

<= Menor ou igual

```
5 <= 5    // true  
5 <= 3    // false
```



Cuidado: comparar texto não é comparar número

Entre textos, `"10" > "9"` é **false**: a comparação é caractere por caractere, como no dicionário.

- O `"1"` vem antes do `"9"`, então o texto `"10"` é considerado "menor".
- Tudo que vem do prompt é texto: converta antes de comparar números.
- Em textos, maiúsculas contam: `"Ana" === "ana"` é **false**.

```
console.log("10" > "9")           // false
console.log(Number("10") > Number("9")) // true
```



Operadores Lógicos

Combinando várias condições em uma só

E, OU e NÃO

&&

And (E)

Devolve **true** só se as duas condições forem verdadeiras.

||

Or (OU)

Devolve **true** se pelo menos uma condição for verdadeira.

!

Not (NÃO)

Inverte o valor: **!true** é **false** e **!false** é **true**.

Tabela verdade: posso ligar o ar?

portaFechada && janelaFechada

porta	janela	&&
true	true	true
true	false	false
false	true	false
false	false	false

estaFrio || estaAbafado

frio	abafado	
true	true	true
true	false	true
false	true	true
false	false	false

No &&, basta um falso para dar falso. No ||, basta um verdadeiro para dar verdadeiro.

Operadores lógicos na prática

```
let portaFechada = true
let janelaFechada = false
let estaFrio = false
let estaAbafado = true

portaFechada && janelaFechada // false
estaFrio || estaAbafado      // true
!estaFrio                     // true

let naoPodeLigar = !(portaFechada && janelaFechada)
// naoPodeLigar = true
```

&& → a janela está aberta, então o resultado é false.

|| → está abafado, então já é true.

! → inverte: **!false** vira true.

!() → os parênteses fazem o **!** inverter o resultado da expressão inteira.

Combinando expressões lógicas

Podemos juntar várias condições em uma só:

```
if ((estaFrio || estaAbafado) && portaFechada && janelaFechada) {  
    ligarArCondicionado()  
} else {  
    desligarArCondicionado()  
}
```

Os parênteses definem a ordem: primeiro o `||`, depois os `&&`. Liga se estiver frio ou abafado, e tudo estiver fechado.

Truthy e falsy

Falsy

Valores que o if trata como falso:

```
false
0
''           // texto vazio
null
undefined
NaN
```

Truthy

Todo o resto é tratado como verdadeiro:

```
true
20
'Ana'
'0'           // é texto!
[]
{}
```

Por isso if (nome) verifica se o usuário digitou algo: se ele só apertar Enter, o texto vem vazio e cai no else.



O que aparece no console?

```
let idade = 18

if (idade = 20) {
  console.log("Tem 20 anos")
} else {
  console.log("Não tem 20 anos")
}
```

Pense antes de rodar: qual é a saída?



Cuidado: = não é o mesmo que ==

Imprime **"Tem 20 anos"** — mesmo a idade sendo 18!

- O `=` sozinho atribui: troca o valor de idade para 20 e devolve 20, que é `truthy`.
- O `==` compara valores (`18 == "18"` é verdadeiro). O `===` compara valor e tipo (`18 === "18"` é falso).
- Dentro de um `if`, use sempre `==` ou `===`. Um `=` ali é quase sempre um bug.

```
if (idade === 20) {    // compara, não altera
```



Hora da Prática

Atividade Prática 1

Condicionais

- Peça ao usuário dois números e mostre qual é o maior deles.
- Use uma condicional com os operadores de comparação ($>$ e $<$).



Cuidado: converta as duas entradas antes de comparar. Como texto, "10" $>$ "9" dá false.



Desafio: trate também o caso em que os dois números são iguais.

Atividade Prática 2

Condicionais

- Desenvolva um programa que peça ao usuário um número e diga se ele é ímpar ou par.



Dica: use o operador módulo (%): se `numero % 2 == 0`, o número é par. Não se esqueça de tratar o 0.

Atividade Prática 3

Conditionais

- Pergunte se o usuário gosta de café.
- Use uma condicional para salvar a resposta em uma variável booleana `gostaDeCafe`.
- Com base nela, crie outra condicional para imprimir uma mensagem personalizada caso ele goste e outra caso não goste.

Atividade Prática 4

Condicionais

- Peça o nome e a idade do usuário e imprima em que ano ele nasceu.
- Para facilitar, crie uma constante com o ano atual: assim não é preciso pedir mais uma entrada.



Cuidado: a idade deve ser um número inteiro. Na mensagem, fale das duas possibilidades: se ele já fez aniversário este ano ou se ainda não fez (um ano a menos).

Atividade Prática 5

Condicionais

- Pergunte o nome e a idade do usuário.
- Se for maior de idade, imprima: "Nome, você já é maior de idade."
- Se for menor, imprima: "Nome, você vai ser maior de idade em X anos." (calcule quantos anos faltam).

Revisão: condicionais e lógica



if / else

Executa um bloco ou outro, dependendo de um teste.

```
if (ok) { ... }  
else { ... }
```



Comparação

Sempre devolve true ou false. Prefira === e !==.

```
idade === 18  
nota >= 7
```



Lógicos

&&: as duas. ||: basta uma. !: inverte.

```
(a || b) && c
```

Dentro de um if, um único = é quase sempre um bug.



Arrays

Guardando vários valores em uma só variável

O que é um Array?



Uma lista de valores

Permite armazenar vários valores em uma única variável.



Qualquer tipo

Números, strings, booleanos, objetos e até outros arrays.



Como um conjunto da matemática

Os elementos ficam entre colchetes `[ ]`, separados por vírgula.



Acessado por índice

Cada elemento tem uma posição, e a contagem começa no 0.

Criando arrays

```
let professores = ['João', 'Edson', 'Jhonatan', 'Eduardo']  
let turmas = ['manhã', 'tarde', 'noite']  
let frutas = ['maçã', 'banana', 'abacaxi']  
let numeros = [10, 20, 30, 40, 50]  
let misturado = ['Ana', 20, true]
```

Colchetes [] abrem e fecham o array. Cada item é separado por vírgula.

Índices: acessando elementos



```
let exemplo = [10, 20, 30, 40, 50]
```

```
console.log(exemplo[0]) // 10
```

```
console.log(exemplo[3]) // 40
```

```
let segundo = exemplo[1] // 20
```

```
console.log(`0 primeiro elemento é ${exemplo[0]}.`)
```



Cuidado: índice que não existe

Acessar uma posição que não existe **não dá erro: devolve **undefined**.**

- O último índice é sempre **length - 1**: num array de 3 itens, os índices vão de 0 a 2.
- Guarde isso para os laços: percorrer com **i <= lista.length** faz a última volta imprimir **undefined**.

```
let frutas = ['maçã', 'banana', 'abacaxi']  
frutas[3] // undefined  
frutas[frutas.length - 1] // 'abacaxi' (o último)
```

Métodos para manipular arrays

push

Adiciona no fim.

pop

Remove o último.

unshift

Adiciona no início.

shift

Remove o primeiro.

splice

Remove itens de uma posição específica.

includes

O item existe? (**true/false**)

indexOf

Em que posição está? (**-1** se não existe)

length

Quantos itens o array tem.

Adicionar e remover nas pontas

```
let personagens = ['Shrek', 'Fiona']

personagens.push('Burro')    // ['Shrek', 'Fiona', 'Burro']
personagens.pop()           // ['Shrek', 'Fiona']

personagens.unshift('Gato')  // ['Gato', 'Shrek', 'Fiona']
personagens.shift()          // ['Shrek', 'Fiona']
```

push e pop trabalham no fim do array; unshift e shift no início.



Cuidado: const com array

O **const** impede **trocar** o array por outro, mas não impede **mudar o que tem dentro**.

- **push**, **pop** e **splice** funcionam normalmente num array criado com **const**.
- O que dá erro é reatribuir: **lista = [...]**. Vale o mesmo para objetos.

```
const lista = ['Shrek']  
lista.push('Fiona')    // ok: ['Shrek', 'Fiona']  
lista = ['Burro']      // TypeError: Assignment to constant variable.
```

splice: removendo do meio

```
let numeros = [10, 20, 30, 40, 50]
numeros.splice(2, 1)           // a partir do índice 2, remove 1 item
console.log(numeros)           // [10, 20, 40, 50]

let personagens = ['Shrek', 'Fiona', 'Príncipe Encantado']
personagens.splice(0, 2)       // a partir do índice 0, remove 2 itens
console.log(personagens)       // ['Príncipe Encantado']
```

splice(posição, quantidade): o primeiro número diz onde começar e o segundo quantos remover.

Consultando: includes, indexOf e length

```
let personagens = ['Príncipe Encantado', 'Fiona']

personagens.includes('Fiona')           // true
personagens.indexOf('Príncipe Encantado') // 0
personagens.indexOf('Shrek')             // -1 (não existe)
personagens.length                       // 2

if (personagens.includes('Príncipe Encantado')) {
  console.log('O filme é Shrek 2 ou 3')
}
```



Hora da Prática

Atividade Prática 1

Arrays

Vamos praticar arrays com uma dinâmica entre vocês! Esta atividade não precisa de entrada do usuário.

- **Crie um array `minhaMesa` com os nomes dos alunos que sentam na sua mesa, incluindo você.**
- **Imprima: "Meu nome é NOME, meus colegas são COLEGA, COLEGA, COLEGA."**
- **Use interpolação e acesse cada nome pela posição no array, da esquerda para a direita.**

Atividade Prática 2

Arrays

- Peça ao usuário duas notas (prova1 e prova2), uma de cada vez.
- Guarde as notas em um array notas e calcule a média usando as posições do array.
- Imprima uma mensagem com o resultado da média.



Dica: crie o array notas vazio, converta cada entrada com `parseFloat()` (trocando vírgula por ponto) e use `push()` para adicionar as notas.

Atividade Prática 3

Arrays

Vamos criar um pequeno gerenciador de tarefas!

- Comece com um array vazio `listaTarefas` e peça ao usuário 3 tarefas do dia, uma de cada vez.
- Imprima: "Você tem X tarefas na sua lista."
- Simule que a última tarefa foi concluída: remova o último item e imprima a lista atualizada.



Dica: use `push()` para adicionar, `.length` para contar e `pop()` para remover. Para imprimir a lista de um jeito mais legível, pesquise o método `join()`.

Revisão: arrays

1 2
3 4

Criar e acessar

Vários valores em uma variável. Índices começam no 0.

```
let lista = [10, 20, 30]
lista[0] // 10
lista[2] // 30
```



Adicionar e remover

Nas pontas ou em uma posição específica.

```
push / unshift
pop / shift
splice(pos, qtd)
```



Consultar

Existe? Onde está?
Quantos são?

```
includes(item)
indexOf(item)
length
```



Objetos

Guardando informações relacionadas com nome

O que é um Objeto?



Informações relacionadas juntas

Permite armazenar e organizar dados que pertencem à mesma "coisa": um carro, uma pessoa.



Pares de chave e valor

Cada propriedade tem um nome (a chave) e um valor associado.



Valores de qualquer tipo

As propriedades podem guardar strings, números, arrays e até funções.



Notação de ponto

Acessamos e alteramos com `objeto.propriedade`.

Cuidado: objeto não é array

12
34

Array: lista numerada

Cada valor é acessado pela posição (índice).

```
let carro = ['Ford', 'Ka', 2009]
carro[1]    // 'Ka'
```



Objeto: lista nomeada

Cada valor é acessado pelo nome da chave.

```
let carro = { modelo: 'Ka' }
carro.modelo // 'Ka'
```

Parecidos, mas com funções diferentes: com um objeto, você não precisa lembrar em que posição cada informação está.

Criando objetos

```
let carro = {  
  fabricante: 'Ford',  
  modelo: 'Ka',  
  ano: 2009  
}  
  
let pessoa = {  
  nome: 'João Pedro dos Santos',  
  idade: 20,  
  cidadeNatal: 'Ponta Grossa'  
}
```

{ } → as chaves abrem e fecham o objeto.

chave: valor → cada propriedade tem um nome, dois-pontos e o valor.

, → a vírgula separa uma propriedade da outra.

nomes → as chaves seguem as regras de variáveis: camelCase, sem acento e sem espaço.

Acessando e alterando valores

```
console.log(carro.ano)           // 2009
console.log(carro['modelo'])     // Ka

let turma = { totalAlunos: 55, horario: 'manhã' }

turma.totalAlunos = turma.totalAlunos - 5
console.log(turma.totalAlunos)  // 50

turma.sala = 'Lab 2'             // cria uma nova propriedade
console.log(turma)
```

Saída no terminal

```
{ totalAlunos: 50, horario: 'manhã', sala: 'Lab 2' }
```



Cuidado ao imprimir objetos

Colocar o objeto inteiro dentro de um texto imprime **[object Object]**.

- Na interpolação, acesse as propriedades: ``${carro.modelo}``.
- Para ver o objeto inteiro, passe ele sozinho: `console.log(carro)`.
- Propriedade que não existe devolve **undefined**, sem erro: confira o nome da chave.

```
console.log(`Carro: ${carro}`)           // Carro: [object Object]
console.log(`Carro: ${carro.modelo}`)     // Carro: Ka
console.log(carro.cor)                    // undefined
```

Objetos podem ter métodos

```
let cachorro = {  
  nome: 'Rex',  
  latir: function () {  
    console.log('Au au!')  
  }  
}  
  
cachorro.latir()    // Au au!
```

latir → uma propriedade cujo valor é uma função: chamamos isso de método.

cachorro.latir() → os parênteses executam o método.

console.log → você já usa um método desde a primeira aula: **log** é um método do objeto **console**!



Hora da Prática

Atividade Prática 1

Objetos

Esta atividade não precisa de entrada do usuário.

- **Crie um objeto com as informações da sua pintura favorita (se ainda não tiver uma, pesquise e escolha).**
- **Ele deve ter as propriedades: nomePintura, nomeArtista, ano e significado (uma frase curta).**
- **Imprima: "Minha pintura favorita é ... feita por ... em ... e significa ...".**

Atividade Prática 2

Objetos

- Crie um objeto livro com as propriedades: titulo, autor, ano e paginas.
- Imprima usando interpolação: "[titulo], de [autor], tem [paginas] páginas."



Dica: acesse cada valor com a notação de ponto, como `livro.titulo`.



Desafio: adicione a propriedade `lido` com valor booleano e imprima se você já leu o livro.

Revisão: objetos



Chave e valor

Informações relacionadas, cada uma com um nome.

```
let carro = {  
  fabricante: 'Ford',  
  modelo: 'Ka'  
}
```



Notação de ponto

No array o acesso é por índice; no objeto, pelo nome.

```
carro.modelo // 'Ka'  
carro.ano = 2009
```



Métodos

Funções guardadas dentro do objeto.

```
cachorro.latir()  
console.log()
```



Laços de Repetição

Fazendo o computador repetir o trabalho por nós

O que são laços de repetição?



Repetir comandos

Executam o mesmo bloco de código várias vezes, sem precisar copiar e colar.



Enquanto a condição for verdadeira

A repetição continua enquanto uma condição for satisfeita e para quando ela fica falsa.



Principais laços

for, while, do...while e forEach.



Muito usados com arrays

Percorrer listas, processar dados e fazer contas sequenciais.

Anatomia do for

```
for (let i = 0; i < 5; i++) {  
  console.log(i)  
}
```

Saída no terminal

```
$ node for.js  
0  
1  
2  
3  
4
```

let i = 0 → inicialização: roda uma única vez, no começo.

i < 5 → condição: testada antes de cada volta. Se for falsa, o laço termina.

i++ → atualização: roda no fim de cada volta.

; → as três partes são separadas por ponto e vírgula, não por vírgula.

Outras formas de escrever o for

```
// atualização com i = i + 1
for (let i = 0; i <= 5; i = i + 1) {
  console.log(i)
}
```

```
// atualização com i += 1
for (let i = 0; i <= 5; i += 1) {
  console.log(i)
}
```

```
// variável declarada fora do laço
let i
for (i = 0; i <= 5; i++) {
  console.log(i)
}
```

i <= 5 → com <=, o laço imprime de 0 a 5: 6 voltas.

i = i + 1 → **i += 1** e **i++** fazem exatamente a mesma coisa.

let i → declarada fora, a variável continua existindo depois do laço.

while

```
// estrutura:  
// while (condição) {  
//   código aqui  
// }  
  
// exemplo  
let horas = 11  
  
while (horas < 12) {  
  console.log('bom dia')  
  horas++  
}
```

Saída no terminal

```
$ node while.js  
bom dia
```

while → repete enquanto a condição for verdadeira.

horas < 12 → é testada antes de cada volta: se já começar falsa, o bloco nem executa.

horas++ → sem isso, a condição nunca fica falsa.

do...while

```
let horas = 12

// não executa nenhuma vez
while (horas < 12) {
  console.log('while')
}

// executa 1 vez
do {
  console.log('do...while')
  horas++
} while (horas < 12)
```

Saída no terminal

```
$ node dowhile.js
do...while
```

do { } → executa o bloco primeiro e testa a condição depois.

diferença → o do...while sempre roda pelo menos uma vez, mesmo com a condição falsa.

forEach: percorrendo um array

```
let horas = [11, 12, 13]

horas.forEach((hora) => {
  console.log(hora)
})
```

Saída no terminal

```
$ node foreach.js
11
12
13
```

forEach → passa por todos os elementos do array, um por um.

(hora) => → uma função curta que recebe cada elemento. Veremos em detalhes na parte de Funções.

for...of: o jeito simples de percorrer

```
let frutas = ['maçã', 'banana', 'abacaxi']  
  
for (let fruta of frutas) {  
  console.log(fruta)  
}
```

Saída no terminal

```
$ node forof.js  
maçã  
banana  
abacaxi
```

for...of → passa por cada elemento do array, sem precisar de índice.

let fruta → a cada volta, recebe o próximo elemento.

quando usar → quando você só precisa dos valores. Se precisar da posição, use o **for** com **i**.

break e continue

```
for (let i = 1; i <= 10; i++) {  
  if (i === 3) continue // pula o 3  
  if (i === 6) break    // para no 6  
  console.log(i)  
}
```

Saída no terminal

```
$ node break.js  
1  
2  
4  
5
```

continue → pula o resto desta volta e vai direto para a próxima.

break → encerra o laço na hora. É o mesmo break que aparece no switch.

uso comum → parar um laço de palpites quando o usuário acerta.



Quantas vezes esse laço executa?

```
let i = 0

while (i < 5) {
  console.log("Rodando...")
}
```

Olhe com atenção o que acontece com a variável i.



Cuidado: laço infinito

Infinitas vezes: o programa trava e você precisa parar com **Ctrl + C** no terminal.

- A variável **i** nunca muda de valor, então **i < 5** é verdadeiro para sempre.
- Todo while precisa de algo que faça a condição virar falsa: **i++** ou **i = i + 1** dentro do bloco.
- No for isso já vem embutido na terceira parte: **for (let i = 0; i < 5; i++)**.

```
while (i < 5) {  
  console.log("Rodando...")  
  i++    // agora o laço termina  
}
```



Hora da Prática

Atividade Prática 1

Laços

- Pergunte ao usuário qual tabuada ele deseja ver e salve o número em uma variável.
- Use um laço for para calcular e mostrar a tabuada de forma formatada no console.



Dica: a condição do for vai de 1 (ou 0) até 10, e a conta é tabuada * i.



Desafio: faça o programa calcular até 100.

Atividade Prática 2

Laços

- Crie um array `listaDeCompras` com quatro itens: "Arroz", "Feijão", "Macarrão" e "Carne".
- Use um laço `for` que vá de 0 até o tamanho do array (usando `.length`).
- Imprima cada item no formato: "Item: [nome do item]".

Atividade Prática 3

Laços

- Crie uma variável `temperaturaAgua` começando em 90.
- Use um laço `while` que se repita enquanto a temperatura for menor que 100.
- Dentro do laço, imprima "A temperatura está em [valor] graus. Aquecendo..." e aumente a temperatura em 2.

Atividade Prática 4

Laços

- Um evento vendeu ingressos em cinco lotes: `lotes = [50, 40, 60, 30, 70]`.
- Crie uma variável `totalIngressos` começando em 0.
- Use um laço `for` para percorrer o array e somar todos os valores em `totalIngressos`.
- Ao final, imprima: "O total de ingressos vendidos foi: `[total]`".

Atividade Prática 5

Laços

Agora vamos trabalhar a lógica!

- Percorra os números de 0 a 999 com um for.
- Some separadamente os pares e os ímpares (se `numero % 2 == 0`, é par) e conte quantos há de cada.
- Mostre as duas somas e os dois totais.



Dica: crie `somaPares`, `somaImpares`, `totalPares` e `totalImpares`. Use `+=` nas somas e `++` nos totais.



Desafio: calcule a média de pares e de ímpares (`somaPares / totalPares`) e imprima qual das duas somas foi maior.

Atividade Prática 6

Laços + Objetos

- Crie um array turma com três objetos, cada um com nome e nota de um aluno.
- Use um laço para percorrer o array e imprimir "Aluno: [nome] — Nota: [nota]".



Dica: dentro do laço, acesse `turma[i].nome` e `turma[i].nota`.



Desafio: calcule e imprima a média das notas da turma ao final.

Revisão: laços de repetição

for

Quando você sabe quantas vezes vai repetir.

```
for (let i = 0; i < 5; i++)
```

while

Repete enquanto a condição for verdadeira; pode nem executar.

```
while (condição) { ... }
```

do...while

Executa pelo menos uma vez antes de testar.

```
do { ... } while (condição)
```

forEach

Percorre todos os elementos de um array.

```
lista.forEach((item) => ...)
```



Switch

Uma alternativa a vários if encadeados

Por que existe o switch?

Quando comparamos a mesma variável com vários valores, os if se acumulam:

```
if (variavel == 1) {  
    // ação 1  
} else if (variavel == 2) {  
    // ação 2  
} else if (variavel == 3) {  
    // ação 3  
} else {  
    // ação geral  
}
```

O switch organiza esse tipo de decisão de um jeito mais limpo.

switch x if

switch

```
switch (variavel) {  
  case 1:  
    // ação 1  
    break  
  case 2:  
    // ação 2  
    break  
  case 3:  
    // ação 3  
    break  
  default:  
    // ação geral  
}
```

if ... else if

```
if (variavel == 1) {  
  // ação 1  
} else if (variavel == 2) {  
  // ação 2  
} else if (variavel == 3) {  
  // ação 3  
} else {  
  // ação geral  
}
```

Cada case é um valor possível; o default é o "senão".

Exemplo: semáforo

switch

```
let cor = 'vermelho'

switch (cor) {
  case 'vermelho':
    console.log('Pare!')
    break
  case 'amarelo':
    console.log('Atenção!')
    break
  case 'verde':
    console.log('Siga!')
    break
  default:
    console.log('Cor desconhecida!')
}
```

if ... else if

```
let cor = 'vermelho'

if (cor === 'vermelho') {
  console.log('Pare!')
} else if (cor === 'amarelo') {
  console.log('Atenção!')
} else if (cor === 'verde') {
  console.log('Siga!')
} else {
  console.log('Cor desconhecida!')
}
```

switch (true): faixas de valores

switch (true)

```
switch (true) {  
  case (nota >= 90 && nota <= 100):  
    console.log("Nota A")  
    break  
  case (nota >= 80 && nota < 90):  
    console.log("Nota B")  
    break  
}
```

if ... else if

```
if (nota >= 90 && nota <= 100) {  
  console.log("Nota A")  
} else if (nota >= 80 && nota < 90) {  
  console.log("Nota B")  
}
```

Com switch (true), entra no primeiro case cuja condição for verdadeira.



O que aparece no console?

```
let cor = 'verde'

switch (cor) {
  case 'verde':
    console.log("Siga!")
  case 'vermelho':
    console.log("Pare!")
}
```

Pense antes de rodar: qual é a saída?



Cuidado: switch sem break não para

Imprime **"Siga!"** e **"Pare!"** — os dois, mesmo a cor sendo verde.

- Quando um case dá certo, o switch continua executando os casos seguintes até encontrar um **break**.
- Isso se chama fall-through e é uma das maiores pegadinhas do switch.
- Coloque **break** ao final de cada case, sempre.



Hora da Prática

Atividade Prática 1

Switch

- Receba do usuário (prompt-sync ou stdin) o gênero do filme: "Ação", "Comédia", "Terror" ou "Animação".
- Use switch para imprimir em qual sala o filme passará: Ação → Sala 1, Comédia → Sala 2, Terror → Sala 3, Animação → Sala 4.
- No default, imprima: "Gênero não encontrado. Verifique as opções válidas."



Dica: use `genero.trim().toLowerCase()` e escreva os cases em minúsculas (case "ação":). Assim, "Ação", "AÇÃO" ou " ação " funcionam.

Atividade Prática 2

Switch

- Receba uma nota de 0 a 100 e, usando switch, devolva a classificação:
- 90 a 100 → "A" · 80 a 89 → "B" · 70 a 79 → "C" · 60 a 69 → "D" · 0 a 59 → "F".
- Caso padrão: "Nota inválida".



Dica: use switch (true) e, em cada caso, condições como `case (nota >= 90 && nota <= 100) :.`



Desafio: peça a nota ao usuário e diga também se ele foi aprovado (C para cima) ou não.

Atividade Prática 3

Switch

- Crie uma variável `codigoProduto` com o valor "A1", "B2" ou "C3".
- Use `switch` para imprimir o salgadinho escolhido: A1 → Batata Chips, B2 → Amendoim, C3 → Biscoito de Chocolate.
- No default: "Código inválido. Tente novamente."

Revisão: switch



Estrutura

Compara a mesma variável com vários valores possíveis.

```
switch (variavel) {  
  case valor:  
    ...  
    break  
  default:  
    ...  
}
```



break

Impede que os casos seguintes executem. Sem ele, acontece o fall-through.



default

O caminho "senão": roda quando nenhum case bateu.



Funções

Escreva uma vez, use quantas vezes quiser

O que é uma função?



Um bloco reutilizável

Agrupar instruções para realizar uma tarefa específica. Você escreve uma vez e "chama" quantas vezes precisar.



Entrada e saída

Recebe dados (parâmetros), processa de forma isolada e devolve um resultado (retorno).



Uma mini-fábrica

Você define como ela funciona uma vez; depois é só enviar os materiais para ela fazer o trabalho pesado.

Estrutura de uma função

```
function nomeDaFuncao(parametros) {  
  // código aqui  
  return resultado  
}  
  
nomeDaFuncao(valores) // chamando a função
```

function → palavra que cria a função.

nomeDaFuncao → segue as regras de variáveis (camelCase). Use um verbo: **calcularMedia**.

(parametros) → os valores que entram. Pode não ter nenhum.

return → o valor que sai. Também é opcional.

nomeDaFuncao() → a função só executa quando é chamada.

Funções com e sem parâmetros

```
function mostrarMensagem() {  
  console.log('Sistema iniciado com sucesso!')  
}  
mostrarMensagem()  
  
function darBoasVindas(nomeUsuario) {  
  console.log('Bem-vindo(a), ' + nomeUsuario + '!')  
}  
darBoasVindas('Ana')  
darBoasVindas('Carlos')
```

Saída no terminal

```
$ node funcoes.js  
Sistema iniciado com sucesso!  
Bem-vindo(a), Ana!  
Bem-vindo(a), Carlos!
```

Função com retorno

```
function calcularDobro(numero) {  
  return numero * 2  
}  
  
let resultado = calcularDobro(15)  
console.log(resultado) // 30
```

return → devolve o valor para quem chamou a função e encerra a execução dela.

let resultado → o valor devolvido pode ser guardado em uma variável e usado depois.

sem return → a função só executa uma ação (como um `console.log`) e devolve **undefined**.



Cuidado: console.log não é return

Uma função sem **return** devolve **undefined**, mesmo que imprima algo na tela.

- **console.log** só mostra o valor para quem está olhando o terminal.
- **return** entrega o valor para o código que chamou a função, para guardar numa variável ou usar em contas.

```
function dobro(n) {  
  console.log(n * 2)    // mostra 10, mas não devolve nada  
}  
let resultado = dobro(5)  
console.log(resultado)  // undefined
```

Arrow Function: a forma moderna



A linguagem evolui

O JavaScript ganhou uma forma mais enxuta e direta de criar funções.



Menos texto, mais ação

A palavra **function** desaparece e dá lugar à "flecha" **=>**.



O resultado é o mesmo

As duas formas fazem exatamente a mesma coisa; é só uma sintaxe mais moderna.

Tradicional x Arrow Function

O jeito tradicional

```
function somar(a, b) {  
  return a + b  
}
```

```
somar(10, 5) // 15
```

Arrow Function

```
const somar = (a, b) => {  
  return a + b  
}
```

```
somar(10, 5) // 15
```

Na arrow function, a função é guardada em uma const, e os parâmetros vêm antes da flecha =>.



Hora da Prática

Atividade Prática 1

Funções

- Crie uma função `calcularMedia` que receba três notas como parâmetros e retorne a média delas.
- Chame a função com as notas 7, 8.5 e 6 e imprima: "A média é [valor]".



Dica: some os três parâmetros, divida por 3 e use `return` para devolver o resultado.



Desafio: faça a função retornar também "Aprovado" (média maior ou igual a 7) ou "Reprovado".

Atividade Prática 2

Funções

- Crie uma função `saudacao` que receba o nome do usuário e imprima "Bem-vindo(a), [nome]!".
- Use o `prompt-sync` para perguntar o nome e passe esse valor para a função.



Dica: a função não precisa de `return`, pois ela mesma faz o `console.log()`.



Desafio: adicione um segundo parâmetro com o período do dia (manhã, tarde ou noite) e monte a saudação completa.

Atividade Prática 3

Funções

- **Crie uma função `verificarIdade` que receba a idade e retorne se a pessoa é maior ou menor de idade.**
- **Peça a idade pelo console e converta a entrada com `parseInt()` antes de passar para a função.**



Dica: use uma condicional dentro da função e `return` para devolver o texto.



Desafio: reescreva a mesma função usando Arrow Function.

Atividade Prática 4

Funções

- Crie uma função `somarLista` que receba um array de números e retorne a soma de todos os elementos.
- Teste com `[10, 20, 30, 40, 50]` e imprima o total no console.



Dica: crie total começando em 0, percorra o array com um laço e vá somando.



Desafio: crie outra função que receba o mesmo array e retorne o maior valor.

Revisão: funções



Declaração

Bloco reutilizável, executado quando é chamado.

```
function nome(params) {  
  ...  
}  
nome(valores)
```



Parâmetros e retorno

Parâmetros entram; o return sai. Sem return, a função só executa uma ação.

```
return a + b
```



Arrow Function

A forma moderna e mais curta.

```
const somar =  
  (a, b) => {  
    return a + b  
  }
```



Debug

Encontrando e corrigindo erros

Três tipos de erro



Sintaxe

O código está escrito errado e nem roda.

Ex.: faltou fechar uma chave ou uma aspa.

SyntaxError



Execução

O código roda, mas quebra no meio.

Ex.: usar uma variável que não existe.

ReferenceError
TypeError



Lógica

O código roda até o fim, mas o resultado está errado.

É o mais difícil: nenhum erro aparece.

4 x 5 = -1 ???

Depurando com console.log

```
function calcularMedia(n1, n2) {  
  let soma = n1 + n2  
  console.log('soma:', soma)    // espiando o valor  
  return soma / 2  
}  
  
console.log(calcularMedia('7', 8))
```

Saída no terminal

```
$ node debug.js  
soma: 78  
39
```

console.log → a forma mais simples de depurar: imprimir o valor no meio do código.

'7' → o log mostra **soma: 78**: o 7 chegou como texto e foi concatenado!

correção → converter a entrada com **parseFloat()** antes da conta.

Usando o debugger do VS Code



1. Breakpoint

Clique à esquerda do número da linha para marcar onde pausar.

2. Iniciar

Menu Run and Debug (**Ctrl + Shift + D**) → Node.js, ou **F5**.

3. Avançar

F10 passa por cima da linha; **F11** entra na função.

4. Observar

Veja os valores em **VARIABLES** e **WATCH** a cada passo.

O Node.js também tem um debugger no terminal: `node inspect arquivo.js`.

Debug com prompt-sync: use o terminal certo



O Debug Console não aceita digitação

Programas que usam prompt travam ou dão erro quando rodam pelo Run and Debug padrão.



Abra o JavaScript Debug Terminal

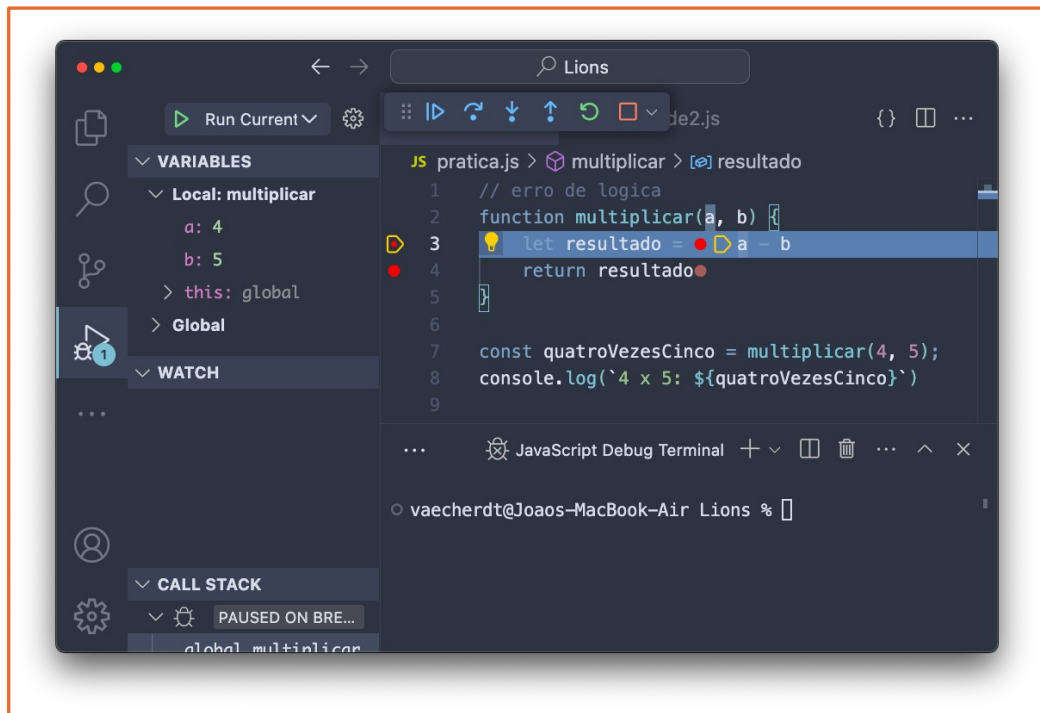
No painel do terminal, clique na setinha ao lado do **+** e escolha JavaScript Debug Terminal (ou **Ctrl + Shift + P** e procure por esse nome).



Rode normalmente

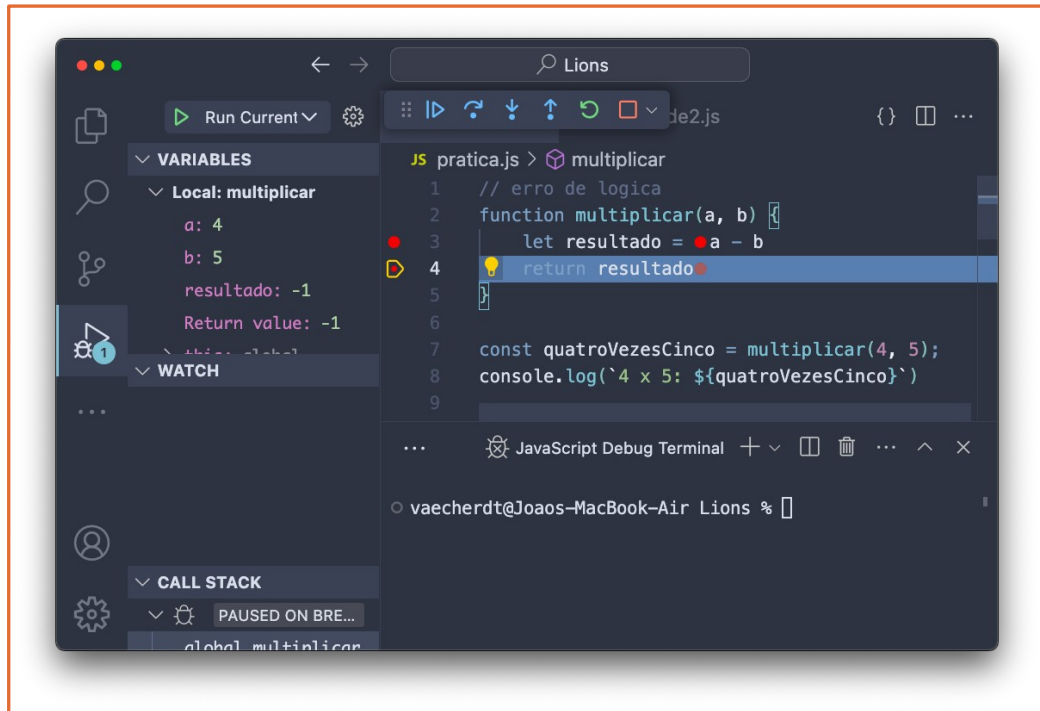
Nele, execute **node arquivo.js**: os breakpoints funcionam e dá para digitar as respostas.

Breakpoint: pausando o código



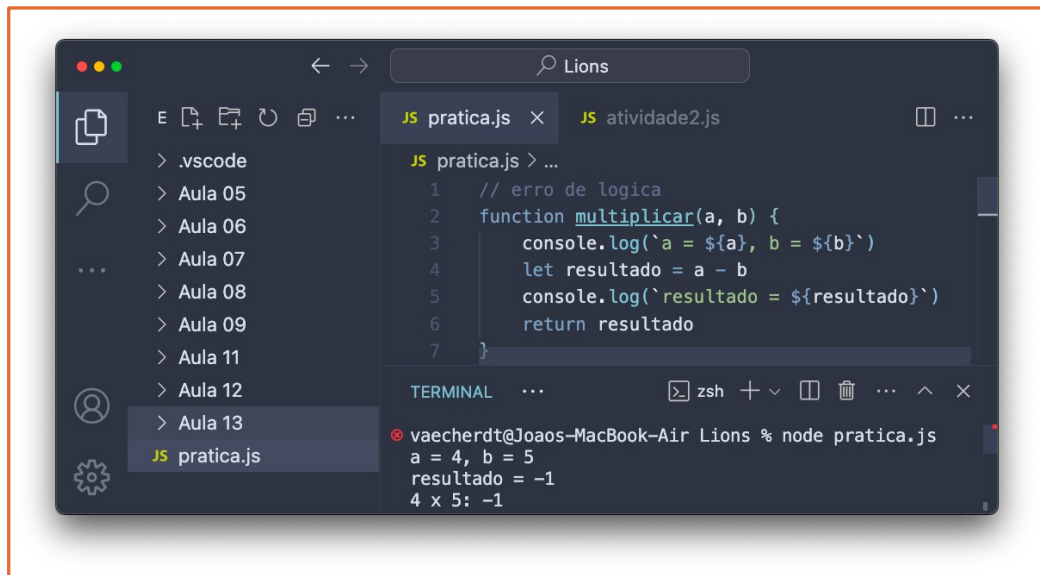
- O ponto vermelho na margem é um breakpoint: a execução pausa naquela linha.
- Em **VARIABLES** você vê o valor real de cada variável no momento da pausa.
- Aqui **a = 4** e **b = 5**: exatamente o que foi passado para **multiplicar(4, 5)**.

Executando passo a passo



- Os botões no topo avançam o código uma linha por vez (step over / step into).
- A cada passo, o painel mostra como as variáveis mudam de valor.
- Repare que **resultado** ficou -1: sinal de que a lógica da conta está errada.

Encontrando o erro de lógica



- O programa não quebrou e não mostrou erro: ele apenas devolveu o valor errado.
- 4 x 5 deveria dar 20, mas o retorno foi -1 porque a função subtrai em vez de multiplicar.
- Esse é o tipo de bug que o debug encontra: o console sozinho não acusa nada.



Ferramentas Úteis

Recursos nativos que vão aparecer bastante daqui pra frente

Comentários



Para humanos, não para máquinas

Anotações para você mesmo no futuro ou para a sua equipe.



O Node.js ignora

Comentários não afetam a velocidade nem o resultado do programa.



// e /* */

// para recados curtos; /* */ para textos longos ou para "desligar" um trecho durante testes.

```
// uma linha
let pontuacao = 100

/* várias
   linhas */

console.log(pontuacao) // fim
```

Biblioteca Math

O **Math** é a biblioteca matemática nativa do JavaScript — muito mais do que números aleatórios.



Constantes

Valores universais como o Pi: **Math.PI**.



Arredondamentos

Mais próximo (**round**), para cima (**ceil**), para baixo (**floor**) ou cortar as decimais (**trunc**).



Potências e raízes

Raiz quadrada (**sqrt**), raiz cúbica (**cbrt**) e potência (**pow**).



Valor absoluto

A distância até o zero, ignorando o sinal: **abs**.

Math na prática

```
console.log(`0 valor de Pi é: ${Math.PI}`) // 3.14159...
```

```
const numeroQuebrado = 4.7
```

```
console.log(Math.round(numeroQuebrado)) // 5 (mais próximo)
```

```
console.log(Math.ceil(numeroQuebrado)) // 5 (sempre para cima)
```

```
console.log(Math.floor(numeroQuebrado)) // 4 (sempre para baixo)
```

```
console.log(Math.trunc(numeroQuebrado)) // 4 (corta a vírgula)
```

```
console.log(Math.sqrt(81)) // 9 (raiz quadrada)
```

```
console.log(Math.cbrt(27)) // 3 (raiz cúbica)
```

```
console.log(Math.pow(2, 3)) // 8 (2 elevado a 3)
```

```
const saldoDevedor = -1500
```

```
console.log(`Você deve: R$ ${Math.abs(saldoDevedor)}`) // R$ 1500
```

Sorteando com Math.random

O `Math.random()` sorteia um decimal entre 0 e 1. Para sortear inteiros, combinamos com outras funções:

```
Math.random()           // 0.7312...   (entre 0 e 1)
Math.random() * 10       // 7.312...   (entre 0 e 10)
Math.floor(Math.random() * 10) // 7      (de 0 a 9)
Math.floor(Math.random() * 10) + 1 // 8      (de 1 a 10)
```

Receita: `Math.floor(Math.random() * N) + 1` sorteia um número inteiro de 1 a N.

Try/Catch



Proteção contra erros

Captura erros e impede que o programa feche sozinho no meio da execução.



try

O bloco com o código que vamos "tentar" executar (e que tem risco de falhar).



catch

Se der erro no try, o programa não quebra: ele cai no catch, onde avisamos o usuário de forma elegante.

Try/Catch na prática

```
try {  
  const palavra = 'LionsDev'  
  palavra.funcaoInexistente() // força um erro  
} catch (erro) {  
  console.log('Ops! Aconteceu um erro, mas o sistema continua.')  
  console.log(`O que falhou: ${erro.message}`)  
}  
  
console.log('O programa chegou ao fim, sem fechar no meio!')
```

Saída no terminal

```
$ node adicional.js  
Ops! Aconteceu um erro, mas o sistema continua.  
O que falhou: palavra.funcaoInexistente is not a function  
O programa chegou ao fim, sem fechar no meio!
```

Controle de tempo



Atrasar ou repetir

Executam código com base no tempo real do relógio, em milissegundos (1000 ms = 1 s).



setTimeout

Executa uma ação uma única vez depois de X milissegundos (ex.: tela de "carregando").



setInterval

Repete uma ação a cada X milissegundos (ex.: um cronômetro). Para parar: `clearInterval`.

setTimeout e setInterval

```
console.log('Preparando o ambiente...')

setTimeout(() => {
  console.log('Ambiente carregado após 3 segundos!')
}, 3000)

let contador = 0
const cronometro = setInterval(() => {
  contador++
  console.log(`Passou ${contador} segundo(s)...`)
  if (contador === 5) {
    clearInterval(cronometro) // para o relógio
    console.log('O tempo esgotou!')
  }
}, 1000)
```

Saída no terminal

```
Preparando o ambiente...
Passou 1 segundo(s)...
Passou 2 segundo(s)...
Ambiente carregado após 3 segundos!
Passou 3 segundo(s)...
Passou 4 segundo(s)...
Passou 5 segundo(s)...
O tempo esgotou!
```

Manipulação de textos

Strings têm métodos nativos (funções prontas) para manipular, limpar e formatar texto.



trim, toLowerCase e toUpperCase

Limpam os espaços das pontas e padronizam tudo em minúsculas ou maiúsculas.



split

Corta uma frase e transforma as palavras em um array.



replace

Substitui uma palavra por outra dentro do texto.

Métodos de string na prática

```
const textoDigitado = '    Sim, Eu Quero APRENDER Node.js!    '  
const textoLimpo = textoDigitado.trim().toLowerCase()  
console.log(`Texto limpo: "${textoLimpo}"`)  
  
const frase = 'A turma é espetacular'  
const palavras = frase.split(' ')  
console.log('Palavras separadas:', palavras)  
  
const fraseAlterada = frase.replace('espetacular', '***')  
console.log(`Frase com censura: ${fraseAlterada}`)
```

Saída no terminal

```
$ node adicional.js  
Texto limpo: "sim, eu quero aprender node.js!"  
Palavras separadas: [ 'A', 'turma', 'é', 'espetacular' ]  
Frase com censura: A turma é ***
```

Módulo de arquivos (fs)



File System

O **fs** é o módulo nativo do Node.js para ler e escrever arquivos no computador.



Os dados não se perdem

O que o programa salva continua lá mesmo depois que você fecha o terminal.



De script a software

Transforma um script temporário em um programa que guarda informações de verdade.

fs na prática

```
const fs = require('fs')

const arquivo = 'tarefas.txt'
const conteudo = '1. Estudar Node.js\n' +
  '2. Praticar lógica\n' +
  '3. Criar projeto final'

fs.writeFileSync(arquivo, conteudo) // cria o arquivo
console.log('Arquivo salvo!')

const textoLido = fs.readFileSync(arquivo, 'utf-8')
console.log('--- Conteúdo lido ---')
console.log(textoLido)
```

Saída no terminal

```
$ node adicional.js
Arquivo salvo!
--- Conteúdo lido ---
1. Estudar Node.js
2. Praticar lógica
3. Criar projeto final
```



Revisão Geral

Como um programa funciona?

Lembra da receita de bolo? O programa é essa sequência de passos.



Entrada

O que o usuário digita ou o que já está nas variáveis.



Algoritmo

A sequência de passos: condicionais, laços, funções...



Saída

O resultado exibido no console.

Relembrando: maior ou menor de idade

```
const prompt = require('prompt-sync')()

let nome = prompt('Como é seu nome? ')
let idade = parseInt(prompt('Qual é sua idade? '))

function processamento(nome, idade) {
  if (idade < 18) {
    let faltam = 18 - idade
    console.log(`${nome}, você será maior de idade em ${faltam} anos.`)
  } else {
    console.log(`${nome}, você já é maior de idade.`)
  }
}

processamento(nome, idade)
```

Cheat sheet — revisão geral

Variáveis

`let, const, var`

Tipos de dados

`string, number,
boolean, object`

Strings

`"a" + b
`Olá ${nome}``

Operadores

`+ - * / %
=== !== > <`

Condicionais

`if (...) { ... }
else { ... }`

Lógicos

`&& || !`

Arrays

`[10, 20, 30]
push pop splice`

Objetos

`{ chave: valor }
obj.propriedade`

Laços

`for / for...of
while / do...while`

Switch

`case valor:
... break`

Funções

`function f() {}
(a, b) => a + b`

Entrada e saída

`prompt('...')
console.log()`



Exercícios de Revisão para Avaliação

Exercício 1

Revisão

Lógica · Par ou ímpar sem %

- **Peça um número ao usuário e descubra se ele é par usando apenas subtrações sucessivas em um laço.**
- **Enquanto o número for maior ou igual a 2, subtraia 2. Ao final, se sobrar 0 é par; se sobrar 1 é ímpar.**



Conteúdos: entrada de dados, laço, condicional e operadores.

Exercício 2

Revisão

Boletim da turma

- Crie um array com 5 notas. Percorra com um laço e calcule a média.
- Ao final, imprima: "Aprovado" (média maior ou igual a 7), "Recuperação" (entre 5 e 7) ou "Reprovado" (abaixo de 5).



Conteúdos: arrays, laço, operadores e condicionais.

Exercício 3

Revisão

Contador de palavras

- Peça uma frase ao usuário e descubra quantas palavras ela tem.



Dica: use `.split(" ")` para transformar a frase em um array e `.length` para contar os itens.



Conteúdos: strings, manipulação de texto, arrays e entrada de dados.

Exercício 4

Revisão

Cadastro de produtos

- **Crie uma função** `cadastrarProduto(nome, preco, quantidade)` **que devolva um objeto com essas três propriedades.**
- **Chame a função três vezes, guarde os objetos em um array e imprima o valor total do estoque (preço x quantidade de cada um, somados).**



Conteúdos: funções com retorno, objetos, arrays e laço.

Exercício 5

Revisão

Lógica · Tabuada com switch

- Peça ao usuário um número de 1 a 5 e, usando switch, imprima a tabuada correspondente com um laço.
- Se o número estiver fora do intervalo, o default deve avisar o usuário.



Conteúdos: switch, laços, entrada de dados e conversão de tipos.

Exercício 6

Revisão

Lógica · Adivinhe o número

- Sorteie um número de 1 a 10 com `Math.floor(Math.random() * 10) + 1`.
- Use um laço para pedir palpites até o usuário acertar, avisando se o palpite foi alto ou baixo. No final, mostre quantas tentativas ele usou.



Dica: use `break` para sair do laço quando acertar, e `sigint: true` no `prompt-sync` para conseguir encerrar com `Ctrl + C`.



Conteúdos: laços, condicionais, variáveis, entrada de dados e biblioteca `Math`.

Material do módulo



Aponte a câmera do celular

[github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo02](https://github.com/nicolassmotta/curso-lions-dev/tree/main/modulo02)



conteudo/
teoria comentada e o cheat sheet do módulo



exercicios_slides/
os exercícios feitos em aula



lista_de_exercicios/
listas para praticar em casa



exercicios_resolvidos/
gabaritos, para conferir depois de tentar



Fim!

Dúvidas? Bora praticar: é programando que se aprende a programar.